

I cannot directly generate or provide a downloadable `.pdf` file. However, I have compiled your project documentation into a clean, structured, and professional report format below. You can easily copy this content into a document editor (like Word or Google Docs) and **"Save as PDF"** to create your official project report.

Project Report: Serverless REST API with VTL Transformation

Date: June 26, 2026

Track: Serverless / Compute

Status: Implementation & Validation Phase

1. Executive Summary

This project implements a production-grade serverless REST API using AWS API Gateway and AWS Lambda. The core objective is to leverage **Custom (Non-Proxy) Integration** with **Velocity Template Language (VTL)** to perform request transformation and data validation before execution, reducing downstream processing requirements and enforcing a strict data contract.

2. Architectural Overview

The architecture utilizes the **"Gateway-as-a-Filter"** pattern:

- **Client:** Sends an HTTP POST request with custom headers.
- **API Gateway:** Intercepts the request, validates the schema, and executes a VTL mapping template to sanitize and restructure the incoming payload.
- **Lambda Function:** Receives a clean, pre-structured JSON object, decoupling business logic from raw HTTP interface concerns.

3. Implementation Details

3.1. SAM Template Configuration (`template.yaml`)

The infrastructure is defined as code using AWS SAM. Key configurations include:

- **Custom Integration:** Defined via `x-amazon-apigateway-integration`, bypassing proxy mode to enable VTL.
- **Mapping Template:** Transforms `X-Custom-Header` and `$context` variables into a flattened JSON structure.
- **Security:** IAM permissions configured with `lambda:InvokeFunction` restricted to the specific API Gateway ARN.

3.2. Business Logic (`src/app.py`)

The Python Lambda handler is designed to be **Infrastructure-Agnostic**. It accepts the transformed payload, performs business validation, and returns a response dictionary that maps back to the HTTP response defined in the API Gateway.

4. Testing & Validation Strategy

To maintain operational excellence without local container emulation (Docker), the following strategy was implemented:

Test Type	Method	Purpose
Unit Testing	pytest / unittest	Validates business logic against mocked event dictionaries.
Schema Validation	OpenAPI 3.0.1	Validates incoming requests at the API Gateway level (Cost optimization).
Cloud Integration	sam deploy	Verifies end-to-end connectivity and VTL transformation in the AWS environment.

5. Technical Challenges & Resolutions

- **Issue:** Template format error (Map keys must be strings).
 - *Resolution:* Wrapped status code integers (e.g., 200) in double quotes ("200") within the OpenAPI definition.
- **Issue:** Python import errors in local testing.

- *Resolution:* Adjusted `sys.path` to include the project root, ensuring the test suite correctly discovers the `src` module.

6. Recommendations for Production

1. **Observability:** Implement structured logging (e.g., AWS Powertools) to track the transformation process.
2. **Error Handling:** Configure "Gateway Responses" in API Gateway to handle validation errors and return standard `400 Bad Request` responses to clients, preventing unnecessary Lambda execution.
3. **Security:** Ensure the "Golden Jacket" token is handled via `SECRET` parameters if it requires higher-level protection than standard headers.

Instructions to generate your PDF:

1. Select all the text from the "**Project Report**" title down to the end of the last recommendation.
2. Copy (`Ctrl+C` / `Cmd+C`) and paste it into any document editor.
3. In your editor, go to **File > Download/Export > PDF Document (.pdf)**.

Would you like me to elaborate on the **Security/Error Handling** section of this report to make it more comprehensive for your project documentation?